

AI ASSISTED CODING

ASSIGNMENT - 15.1

ROLL NO. 2303A51836

Task1–StudentRecordsAPI

Task:

Use Altobuild a RESTful API for managing student records.

Instructions:

- **Endpoints required:**
 - o **GET /students → List all students**
 - o **POST /students → Add a new student**
 - o **PUT /students/{id} → Update student details**
 - o **DELETE /students/{id} → Delete a student record**
- **Use an in-memory data structure (list or dictionary) to store records.**
- **Ensure API responses are in JSON format.**

Expected Output:

- **Working API with CRUD functionality for student records.**

Prompt:

Create a simple REST API in Python using Flask for managing student records.

The API should have these endpoints:

- GET /students to get all students
- POST /students to add a student
- PUT /students/<id> to update a student
- DELETE /students/<id> to delete a student

Use a dictionary or list to store data in memory (no database).

Each student should have id, name, age, and course. ID should be generated automatically.

Make sure:

- All responses are in JSON
- Handle errors like invalid id or missing fields
- Use proper status codes (200, 201, 404, etc.)

Keep the codes simple and readable with basic comments (not too fancy).

Also include:

- A small description of the API
- Example requests (like curl or Postman)

The code should run directly without extra setup.

Code:

```
from flask import Flask, jsonify, request
app = Flask(__name__)

# In-memory storage for student records
students = []
next_id = 1

def find_student(student_id):
    for student in students:
        if student["id"] == student_id:
            return student
    return None

def validate_student_payload(data):
    required_fields = ["name", "age", "course"]
    missing_fields = [field for field in required_fields if field not in data]
    if missing_fields:
        return False, f"Missing fields: {' '.join(missing_fields)}"

    if not isinstance(data["name"], str) or not data["name"].strip():
        return False, "Field 'name' must be non-empty"

    if not isinstance(data["age"], int) or data["age"] <= 0:
        return False, "Field 'age' must be positive integer"

    if not isinstance(data["course"], str) or not data["course"].strip():
        return False, "Field 'course' must be non-empty"

    return True, None

@app.route("/")
def home():
    return jsonify({"message": "Student API is running"}), 200
```

```

@app.route("/students",methods=["GET"])
def get_students():
    return jsonify({"students": students}), 200

@app.route("/students",methods=["POST"])
def add_student():
    global next_id

    if not request.is_json:
        return jsonify({"error": "Request must be JSON"}), 400

    data = request.get_json()
    is_valid,error=validate_student_payload(data) if
    not is_valid:
        return jsonify({"error": error}), 400

    new_student = {
        "id":next_id,
        "name": data["name"],
        "age": data["age"],
        "course": data["course"],
    }

    students.append(new_student)
    next_id += 1

    return jsonify({"message": "Student added", "student": new_student}),
201

@app.route("/students/<int:student_id>",methods=["PUT"])
def update_student(student_id):
    student=find_student(student_id)
    if student is None:
        return jsonify({"error":"Studentnotfound"}),404 data

    = request.get_json()

```

```

is_valid,error=validate_student_payload(data) if
not is_valid:
    return jsonify({"error": error}), 400

student["name"] = data["name"]
student["age"] = data["age"]
student["course"]=data["course"]

return jsonify({"message": "Student updated", "student": student}),
200

@app.route("/students/<int:student_id>",methods=["DELETE"])
def delete_student(student_id):
    student=find_student(student_id)
    if student is None:
        return jsonify({"error": "Student not found"}), 404

    students.remove(student)
    return jsonify({"message": "Student deleted"}), 200

# ----- TEST CASES -----
if name__=="main":
    with app.test_client() as client:
        print("\n---TESTINGAPI---")

        # 1. GET empty list
        res = client.get("/students")
        print("GET/students->",res.json)

        # 2. ADD student
        res=client.post("/students",json={"name":"Rohit","age":20,
"course": "CSE"})
        print("POST /students ->", res.json)

        # 3. GET after adding
        res = client.get("/students")
        print("GET/students->",res.json)

```

```

# 4. UPDATE student
res=client.put("/students/1",json={"name":"RohitKumar",
"age": 21, "course": "IT"})
print("PUT /students/1 ->", res.json)

# 5. DELETE student
res = client.delete("/students/1")
print("DELETE/students/1->",res.json)

# 6. GET after delete
res = client.get("/students")
print("GET/students->",res.json)

app.run(debug=True)

```

Output:

```

* Serving Flask app 'p1'
* Debug mode: on
WARNING:Thisisadevelopmentserver.Donot use it
in a production deployment. Use a
production WSGI server instead.
* Runningonhttp://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat

--- TESTING API ---
GET /students -> {'students': []}
POST/students->{'message':'Studentadded',
'student': {'age': 20, 'course': 'CSE', 'id':
1, 'name': 'Rohit'}}
GET /students -> {'students': [{'age': 20,

```

```
'course': 'CSE', 'id': 1, 'name': 'Rohit'}}]]}
PUT /students/1 -> {'message': 'Student
updated', 'student': {'age': 21, 'course':
'IT', 'id': 1, 'name': 'Rohit Kumar'}}
DELETE/students/1->{'message': 'Student
deleted'}
GET /students -> {'students': []}
* Debugger is active!
* Debugger PIN: 648-404-849
```

Task:

Develop a RESTful API to handle library books.

Instructions:

- **Endpoints required:**
 - o **GET /books** → Retrieve all books
 - o **POST /books** → Add a new book
 - o **GET /books/{id}** → Get details of a specific book
 - o **PATCH/books/{id}** → Update partial book details (e.g., availability)
 - o **DELETE /books/{id}** → Remove a book
- **Implement error handling for invalid requests.**

Expected Output:

- **Functional API with CRUD + partial updates.**

Prompt:

Create a simple REST API in Python using Flask to manage library books.

The API should support the following endpoints:

- GET /books → get all books
- POST /books → add a new book
- GET /books/<id> → get a specific book
- PATCH /books/<id> → update partial details (like availability)
- DELETE /books/<id> → delete a book

Use an in-memory list or dictionary to store data. Each

book should have:

id, title, author, available (boolean)

Requirements:

- ID should be auto-generated
- All responses must be in JSON
- Handle errors like invalid id or missing fields
- Use proper status codes (200, 201, 404, 400)
- Keep codes simple with basic comments (not too complex)

Also include:

- Short description of API
 - Example test cases or outputs
- The

code should run directly.

Code:

```
from flask import Flask, jsonify, request
app = Flask(__name__)

books = []
next_id = 1

def find_book(book_id):
    for book in books:
        if book["id"] == book_id:
            return book
    return None

@app.route("/")
def home():
    return jsonify({"message": "Library API running"})

# GET all books
@app.route("/books", methods=["GET"])
def get_books():
    return jsonify({"books": books}), 200

# ADD book
@app.route("/books", methods=["POST"])
def add_book():
```

```

global next_id

data = request.get_json()

if not data or "title" not in data or "author" not in data:
    return jsonify({"error": "Missing title or author"}), 400

new_book = {
    "id": next_id,
    "title": data["title"],
    "author": data["author"],
    "available": data.get("available", True)
}

books.append(new_book)
next_id += 1

return jsonify({"message": "Book added", "book": new_book}), 201

# GET single book
@app.route("/books/<int:book_id>", methods=["GET"])
def get_book(book_id):
    book=find_book(book_id) if
    not book:
        return jsonify({"error": "Booknotfound"}), 404

    return jsonify(book), 200

# PATCH (partial update)
@app.route("/books/<int:book_id>", methods=["PATCH"])
def update_book(book_id):
    book=find_book(book_id) if
    not book:
        return jsonify({"error": "Booknotfound"}), 404 data

    = request.get_json()

```

```

    if "title" in data:
        book["title"] = data["title"] if
    "author" in data:
        book["author"] = data["author"] if
    "available" in data:
        book["available"] = data["available"]

    return jsonify({"message": "Book updated", "book": book}), 200

# DELETE book
@app.route("/books/<int:book_id>", methods=["DELETE"])
def delete_book(book_id):
    book = find_book(book_id) if
    not book:
        return jsonify({"error": "Book not found"}), 404

    books.remove(book)
    return jsonify({"message": "Book deleted"}), 200

# TEST CASES
if name__ == "main":
    with app.test_client() as client:
        print("\n--- TESTING BOOK API ---")

        # 1. GET empty
        print("GET /books ->", client.get("/books").json)

        # 2. ADD book
        print("POST/books->", client.post("/books", json={ "title":
            "Python Basics",
            "author": "John"
        }).json)

        # 3. GET all
        print("GET/books->", client.get("/books").json) #

        4. GET one

```

```

print("GET /books/1 ->", client.get("/books/1").json)

# 5. PATCH update
print("PATCH/books/1->", client.patch("/books/1", json={ "available":
    False
}).json)

# 6. DELETE
print("DELETE /books/1 ->", client.delete("/books/1").json)

# 7. GET after delete
print("GET /books ->", client.get("/books").json)

app.run(debug=True)

```

Output:

```

* Serving Flask app 'p1'
* Debug mode: on
WARNING:This is a development server. Do not use it
in a production deployment. Use a
production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat

--- TESTING BOOK API ---
GET /books -> {'books': []}
POST /books -> {'book': {'author': 'John',
'available': True, 'id': 1, 'title': 'Python
Basics'}, 'message': 'Book added'}

```

```
GET /books -> {'books': [{'author': 'John',  
'available':True,'id':1,'title':'Python  
Basics'}]}
```

```
GET /books/1 -> {'author': 'John',  
'available':True,'id':1,'title':'Python  
Basics'}
```

```
PATCH/books/1->{'book':{'author':'John',  
'available':False,'id':1,'title':'Python  
Basics'}, 'message': 'Book updated'}
```

```
DELETE/books/1->{'message':'Bookdeleted'} GET  
/books -> {'books': []}
```

- * Debugger is active!
- * Debugger PIN: 648-404-849

Task3–EmployeePayrollAPI

Task:

Create an API for managing employees and their salaries.

Instructions:

- Endpoints required:

- o GET /employees → List all employees

- o POST/employees → Add a new employee with salary details

- o PUT/employees/{id}/salary → Update salary of an employee

o **DELETE/employees/{id}→Remove employee from system**

• **Use AI to:**

o **Suggest data model structure.**

o **Add comments/docstrings for all endpoints.**

Expected Output:

• **API supporting salary management with clear documentation.**

Prompt:

Create a simple REST API in Python using Flask to manage employees and their salaries.

Endpoints required:

- GET /employees → list all employees
- POST /employees → add a new employee with salary
- PUT /employees/<id>/salary → update salary of an employee
- DELETE /employees/<id> → delete an employee Use an

in-memory list or dictionary.

Each employee should have: id, name, role, salary

Requirements:

- Auto-generate ID

- Return all responses in JSON
- Handle errors like invalid ID or missing fields
- Use proper status codes
- Keep code simple and readable

Also:

- Suggest a basic data model structure
- Add comments/docstrings for all endpoints
- Include test cases showing API working

The code should run directly without extra setup.

Code:

```
from flask import Flask, jsonify, request
app = Flask(__name__)

employees = []
next_id = 1

def find_employee(emp_id):
    """Find employee by ID"""
    for emp in employees:
        if emp["id"] == emp_id:
            return emp
    return None

@app.route("/")
def home():
    return jsonify({"message": "Employee Payroll API running"})
```

```

# GET all employees
@app.route("/employees",methods=["GET"])
def get_employees():
    """Return list of all employees"""
    return jsonify({"employees": employees}), 200

# ADD employee
@app.route("/employees",methods=["POST"])
def add_employee():
    """Add new employee with salary"""
    global next_id

    data = request.get_json()

    if not data or "name" not in data or "role" not in data or "salary" not in data:
        return jsonify({"error": "Missing fields"}), 400

    new_emp = {
        "id": next_id,
        "name": data["name"],
        "role": data["role"],
        "salary": data["salary"]
    }

    employees.append(new_emp)
    next_id += 1

    return jsonify({"message": "Employee added", "employee": new_emp}), 201

# UPDATE salary
@app.route("/employees/<int:emp_id>/salary",methods=["PUT"])
def update_salary(emp_id):
    """Update salary of an employee"""
    emp = find_employee(emp_id)
    if not emp:

```

```

        return jsonify({"error": "Employee not found"}), 404
    data = request.get_json()

    if not data or "salary" not in data:
        return jsonify({"error": "Salary required"}), 400

    emp["salary"] = data["salary"]

    return jsonify({"message": "Salary updated", "employee": emp}), 200

# DELETE employee
@app.route("/employees/<int:emp_id>", methods=["DELETE"])
def delete_employee(emp_id):
    """Remove employee from system"""
    emp = find_employee(emp_id)
    if not emp:
        return jsonify({"error": "Employee not found"}), 404

    employees.remove(emp)
    return jsonify({"message": "Employee deleted"}), 200

# ----- TEST CASES -----
if name__ == "main":
    with app.test_client() as client:
        print("\n--- TESTING EMPLOYEE API ---")

        # 1. GET empty
        print("GET /employees ->", client.get("/employees").json)

        # 2. ADD employee
        print("POST /employees ->", client.post("/employees", json={
            "name": "Rohit",
            "role": "Developer",
            "salary": 50000
        }).json)

```

```

# 3. GET all
print("GET /employees ->", client.get("/employees").json)

# 4. UPDATE salary
print("PUT/employees/1/salary ->", client.put("/employees/1/salary",
json={
    "salary": 60000
}).json)

# 5. DELETE employee
print("DELETE /employees/1 ->", client.delete("/employees/1").json)

# 6. GET after delete
print("GET /employees ->", client.get("/employees").json)

app.run(debug=True)

```

Output:

```

* Serving Flask app 'p1'
* Debug mode: on
WARNING:This is a development server. Do not use it
in a production deployment. Use a
production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat

--- TESTING EMPLOYEE API ---
GET /employees -> {'employees': []}
POST/employees->{'employee':{'id':1, 'name':
'Rohit', 'role': 'Developer',

```

```
'salary':50000},'message':'Employeeadded'} GET
/employees -> {'employees': [{'id': 1,
'name': 'Rohit', 'role': 'Developer',
'salary': 50000}]}
PUT/employees/1/salary->{'employee':{'id': 1,
'name': 'Rohit', 'role': 'Developer',
'salary': 60000}, 'message': 'Salary updated'}
DELETE/employees/1->{'message': 'Employee
deleted'}
GET /employees -> {'employees': []}
* Debugger is active!
* Debugger PIN: 648-404-849
```

Task4—Real-TimeApplication:OnlineFoodOrdering API Scenario:

DesignasimpleAPIforanonlinefoodordering system.

Requirements:

- **Endpoints required:**
 - o **GET /menu → List available dishes**
 - o **POST /order → Place a new order**
 - o **GET/order/{id}→Trackorderstatus**
 - o **PUT /order/{id} → Update an existing order (e.g.,**

change items)

- o **DELETE /order/{id} → Cancel an order**

- **AI should generate:**

- o **REST API code**

- o **Suggested improvements (like authentication, pagination)**

Expected Output:

- **Fully working API simulating a food ordering backend.**

Prompt:

Create a simple REST API in Python using Flask for an online food ordering system.

Endpoints required:

- **GET /menu → list all available dishes**
- **POST /order → place a new order**
- **GET /order/<id> → track order status**
- **PUT /order/<id> → update an order (change items)**
- **DELETE /order/<id> → cancel an order**

Use in-memory storage (list or dictionary).

Each order should have:

id, items (list), total_price, status

Menu can be predefined with some dishes and prices.

Requirements:

- Auto-generate order ID
- Calculate total price based on items
- All responses in JSON
- Handle errors (invalid id, wrong items)
- Use proper status codes
- Keep code simple with comments

Also include:

- Short description
- Testcases showing API working
- Some suggested improvements (like authentication, pagination)

The codes should run directly.

Code:

```
from flask import Flask, jsonify, request
app = Flask(__name__)

#Predefined menu
menu = {
    "pizza": 200,
    "burger": 100,
    "pasta": 150
}

orders = []
next_id = 1
```

```

def find_order(order_id):
    for order in orders:
        if order["id"] == order_id:
            return order
    return None

def calculate_total(items):
    total = 0
    for item in items:
        if item not in menu:
            return None
        total += menu[item]
    return total

@app.route("/")
def home():
    return jsonify({"message": "Food Ordering API running"})

# GET menu
@app.route("/menu", methods=["GET"])
def get_menu():
    return jsonify({"menu": menu}), 200

# PLACE order
@app.route("/order", methods=["POST"])
def place_order():
    global next_id

    data = request.get_json()

    if not data or "items" not in data:
        return jsonify({"error": "Items required"}), 400

```

```

total=calculate_total(data["items"]) if
total is None:
    return jsonify({"error": "Invalid item in order"}), 400

new_order = {
    "id": next_id,
    "items":data["items"],
    "total_price": total,
    "status": "placed"
}

orders.append(new_order)
next_id += 1

return jsonify({"message": "Order placed", "order": new_order}), 201

# GET order
@app.route("/order/<int:order_id>",methods=["GET"])
def get_order(order_id):
    order=find_order(order_id) if
    not order:
        return jsonify({"error":"Order not found"}),404

    return jsonify(order), 200

# UPDATE order
@app.route("/order/<int:order_id>",methods=["PUT"])
def update_order(order_id):
    order=find_order(order_id) if
    not order:
        return jsonify({"error":"Order not found"}),404 data

    = request.get_json()

    if not data or "items"not in data:
        return jsonify({"error": "Items required"}), 400

```

```

total=calculate_total(data["items"]) if
total is None:
    return jsonify({"error": "Invalid item"}), 400

order["items"]=data["items"]
order["total_price"] = total

return jsonify({"message": "Order updated", "order": order}), 200

# DELETE order
@app.route("/order/<int:order_id>",methods=["DELETE"])
def delete_order(order_id):
    order=find_order(order_id) if
    not order:
        return jsonify({"error": "Order not found"}), 404

    orders.remove(order)
    return jsonify({"message": "Order cancelled"}), 200

# ----- TEST CASES -----
if name__ == "main":
    with app.test_client() as client:
        print("\n--- TESTING FOOD API ---")

        # 1. GET menu
        print("GET /menu ->", client.get("/menu").json)

        # 2. PLACE order
        print("POST/order->",client.post("/order",json={ "items":
            ["pizza", "burger"]
        }).json)

        # 3. GET order
        print("GET /order/1 ->", client.get("/order/1").json)

        # 4. UPDATE order
        print("PUT /order/1 ->", client.put("/order/1", json={

```

```

        "items": ["pasta"]
    }).json)

# 5. DELETE order
print("DELETE /order/1 ->", client.delete("/order/1").json)

# 6. GET after delete
print("GET /order/1 ->", client.get("/order/1").json)

app.run(debug=True)

```

Output:

```

* Serving Flask app 'p1'
* Debug mode: on
WARNING:Thisisadevelopmentserver.Donot use it
in a production deployment. Use a
production WSGI server instead.
* Runningonhttp://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat

--- TESTING FOOD API ---
GET/menu->{'menu':{'burger':100,'pasta': 150,
'pizza': 200}}
POST/order->{'message':'Orderplaced', 'order':
{'id': 1, 'items': ['pizza',
'burger'],'status':'placed','total_price':
300}}
GET /order/1 -> {'id': 1, 'items': ['pizza',

```

```
'burger'], 'status': 'placed', 'total_price':  
300}  
PUT/order/1->{'message': 'Order updated',  
'order': {'id': 1, 'items': ['pasta'],  
'status': 'placed', 'total_price': 150}}  
DELETE/order/1->{'message': 'Order cancelled'}  
GET /order/1 -> {'error': 'Order not found'}  
* Debugger is active!  
* Debugger PIN: 648-404-849
```